CSED233 · 데이터구조 · 포스텍

Ch4. List, Stack, Queue — 선형 자료구조

1. List (리스트)

정의 & 개념

- **List(리스트)**: 원소의 중복을 허용하는, 유한하고 순서 있는 수열

$$L = \langle a_1, a_2, \ldots, a_n \rangle$$

- n = length (size), $n = 0$이면 empty list
- **Head** (front) ↔ **Tail** (rear)
- "current position": 원소 사이 fence로 표시
  - 예: $\langle 20, 23 | 12, 15 \rangle$ 에서 current에 10 insert → $\langle 20, 23, 10, 12, 15 \rangle$

Sorted vs Ordered (중요한 구분)

- **Ordered**: 원소 간 **순서(position)** 가 정의됨 — 예: $\langle 20, 23, 12, 15 \rangle$ (위치 있음, 크기 무관)
- **Sorted**: key 값 기준 정렬 — 예: $\langle 12, 15, 20, 23 \rangle$
- → **모든 List는 ordered지만, sorted는 아닐 수 있다.**

표준 Operations

Op	의미
Insert(x, p, L)	위치 p에 x 삽입
Delete(p, L)	위치 p 원소 삭제
Next(p, L)	p 다음 위치 포인터
Previous(p, L)	p 이전 위치 포인터

Locate(x, L)	x의 위치 찾기
Retrieve(p, L)	p에서 원소 값
MakeNull(L)	empty로 만들기
First(L)	첫 위치
END(L)	마지막 다음(past-end)

2. Array-Based Implementation

구조

```
char L[max];
int last = 현재 원소 개수
int p    = position (index)
```

Insert(x, 2, L) 동작

```
초기: [a][b][c][d][_][_]    last=4
삽입: [a][x][b][c][d][_]    last=5
```

- p=2에 x 넣으려면 **p..last 전부 오른쪽으로 한 칸 shift** 필요
- **Running time =** $O(n)$ (linear, worst case 모든 원소 이동)
- Delete, Locate도 마찬가지 → $O(n)$

3. Pointer-Based (Linked List(연결 리스트))

Singly-Linked List(단일 연결 리스트)

```
struct cell {
  char info;
  cell* link;
};
```

- **Node = info + link**

- **L → [a] → [b] → [c] → [d] → NULL**

Fence(펜스 · 현재 위치 표시) (current position) 표현

Option 1: `p`가 현재 원소 **직접** 가리킴

- 문제: Insert 시 선행 노드의 link를 바꿔야 하는데 직접 접근 불가 → **선행 노드 access 불편**

Option 2 (One-step-ahead convention): `p`가 **선행 원소**를 가리킴

- `p`의 정의가 달라짐: `p`는 "b" 노드 → current는 "c" 의미
- → Insert 쉬움

Empty list / 왼쪽 partition 빈 경우 처리

- **Solution 1**: if로 special case
- **Solution 2 (권장)**: **Dummy header(더미 헤더) node** 도입
  - `L → [dummy] → [a] → [b] → ...`
  - 모든 경우 일관 처리 → 코드 단순화

Q1. Insert(x, p, L)의 시간복잡도는?

> **힌트**

연결 리스트에서 위치 p의 포인터가 이미 주어지면 탐색이 필요 없다.

> **정답**

개념은 힌트 참고.

포인터 조작만 하면 되므로 $O(1)$ — 탐색 비용 없음.

4. Doubly-Linked List(이중 연결 리스트)

구조

```
struct cell {
  char info;
  cell* prev;
```

```
  cell* next;
};
```

- 양방향 탐색 가능, 선행 노드 접근 $O(1)$

5. Cursor-Based (Simulated Pointer(포인터))

개념

- **배열 index로 pointer simulate**
- 한 배열에 여러 list 저장 가능
- Link = 0 → list tail

예시: 두 리스트 저장

index	info	link
1	c	0
2	a	4
3	y	0
4	b	1
5	-	-
6	x	3

- Head of L_1 = 2 (a → b → c → 0)
- Head of L_2 = 6 (x → y → 0)

시간복잡도:

- Insert/Delete: $O(1)$ (link 재조정만)
- Retrieve (특정 위치): $O(n)$

6. Stack(스택) (LIFO(후입선출))

정의

- **Last-In, First-Out**: 모든 insert/delete가 **한 끝 (Top)** 에서 발생

$$S = \langle a_1, a_2, \ldots, a_n \rangle,\ \mathrm{Top} = a_1$$

- 어떤 list 구현(pointer/array/cursor)으로도 구현 가능

Operations

- Push(x, S): $S = \langle x, a_1, \ldots, a_n \rangle$
- Pop(S): $S = \langle a_2, \ldots, a_n \rangle$
- Top(S): 첫 원소 반환
- MakeNull(S), IsEmpty(S)

Stack(스택) 구현

Array-based:

- Top 위치를 l로 잡으면: push/pop $O(1)$
- Top을 고정 위치 1로 잡으면: push마다 shift → $O(n)$
- 한 배열에 **2개 stack 저장** 가능 (양끝에서 서로 마주보고 grow)

Linked stack: pointer-based list와 거의 동일. 대표 예: call stack.

---

7. Queue(큐) (FIFO(선입선출))

정의

- **First-In First-Out**
- **Front** (삭제 위치), **Rear** (삽입 위치)

Operations

- Enqueue(x, Q): rear에 x 추가
- Dequeue(Q): front 원소 제거
- Front(Q): front 원소 반환
- MakeNull(Q), IsEmpty(Q)

8. Array-Based Queue(큐): Drifting Problem

문제 상황

```
size 6 array, enqueue a,b,c → [a][b][c][_][_][_]   F=0 R=2
dequeue twice               → [_][_][c][_][_][_]   F=2 R=2
enqueue d,e,f               → [_][_][c][d][e][f]   F=2 R=5
```

- **뒤 공간 없음** → 앞은 비었는데 enqueue 불가
- 이것을 **"drifting queue"** 문제라고 함

해결: Circular Array (환형 배열)

- **modulo 연산**: $n + 1 \equiv 1 \pmod{n}$
- 마지막 인덱스 → 첫 인덱스로 수학적 연결

```
enqueue(x):
 R = (R+1) % size
 array[R] = x
```

Empty vs Full 구별 문제

- $R + 1 \equiv F \pmod{\text{size}}$ 조건이 **empty**인지 **full**인지 모호
- $F = R$ 도 마찬가지

해결책:

1. **Explicit count variable** — enqueue → cnt++, dequeue → cnt--, empty ⟺ cnt=0
2. **Boolean flag** empty
3. 한 slot을 **낭비**해서 full 조건 다르게 (= 원형 큐 표준 기법)

9. Linked Queue(큐)

두 pointer F, R (dummy header 없음)

```
F → [a] → [b] → [c] ← R
```

- Empty(Q) ⟺ F = R = NULL

- **Enqueue(x)**: 새 노드 만들고 R->next = 새노드; R = 새노드;
- 빈 큐에 삽입 시 **같은 코드로 처리 안 됨** → special case 필요

Dummy Header 버전

```
F → [dummy] → [a] → [b] → [c] ← R
```

- Empty: F = R = dummy
- 모든 경우 일관 처리 → 코드 간결
- 단, 메모리 dummy 1개 추가 낭비

10. Queue(큐) Variants

- **Queue**: FIFO 기본
- **Circular queue(환형 큐)**: 환형 배열 기반
- **Priority queue**: 우선순위 기반 (Ch7)
- **Double-ended queue (deque)**: 양끝에서 insert/delete 모두 가능

시간복잡도 정리

구현	Insert	Delete	Search	비고
Array(배열) list	$O(n)$	$O(n)$	$O(n)$	shift 필요
Linked list	$O(1)$*	$O(1)$*	$O(n)$	*삽입·삭제 위치의 선행 노드 포인터가 주어질 때 (위치 탐색 비용 별도)
Cursor(커서) list	$O(1)$	$O(1)$	$O(n)$	
Array(배열) stack	$O(1)$	$O(1)$	—	
Linked stack	$O(1)$	$O(1)$	—	
Circular queue(환형 큐)	$O(1)$	$O(1)$	—	

Linked queue	$O(1)$	$O(1)$	—